ONLINE RETAIL CUSTOMER CHURN

Imagine you are the Head of Data Science for a large online store. Thousands of people visit your website, buy products, and interact with your brand every day. However, you have a problem: some of your customers are leaving and never coming back. In the business world, when a customer stops doing business with a company, it is called Customer Churn. It is much more expensive to find new customers than it is to keep existing ones, so preventing churn is a million-dollar problem.

This project *Online Retail Customer Churn* is a snapshot of a customer base situation. It contains information about who the customers are, how they spend their money, and how they interact with the business.

The Target Variable
Target_Churn: This is the most important column in the entire dataset used for this project. It is a simple True or False.
True = The customer churned (they left us).
False = The customer stayed.

The Machine Learning Mission:
As a data scientist my goal was to use Logistic Regression to find the hidden patterns in this data.
I wanted the math to figure out things like: "If a customer has a low Satisfaction Score and a high Number of Support Contacts, are they more likely to churn?" By training this model on this historical data, I eventually fed it information about current customers and predicted who was about to leave – allowing the marketing team to step in and save the relationship before it was too late.

Three (3) phases were involved in the project:

Phase 1: Data Preprocessing & Feature Engineering

- **Data Ingestion:** I loaded the provided dataset.
- **Target Definition:** I isolated Target_Churn as the dependent variable.
- **Categorical Encoding:** I applied One-Hot Encoding strictly using a drop_first=True parameter to avoid the dummy variable trap. This was applied to the following categorical variables:Gender, Email_Opt_In, Promotion_Response, Target_Churn
- **Boolean/Numeric Processing:** I ensured all remaining numerical and boolean features were properly scaled or formatted for ingestion by a linear model.

Phase 2: Model Training

- **Data Splitting:** I implemented a reproducible train-test split and partitioned the dataset into 80% training data and 20% testing data.
- **Algorithm Selection**: I trained a standard Logistic Regression model on the 80% training subset.

Phase 3: Evaluation & Visual Reporting

- **Metrics:** I generated standard classification metrics (Accuracy, Precision, Recall, F1-Score).
- **Visualizations:** I constructed clear, highly readable visual reports to aid student comprehension. The required visuals included Confusion Matrix, ROC Curve / AUC and Feature Importance / Coefficient Plot

PHASE 1: DATA PREPROCESSING & FEATURE ENGINEERING

```
## Importing Neccessary Libraries

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix, roc_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns

## 1. Data Ingestion
# Load the dataset. We drop 'Customer_ID' because it is a random identifier
# and has no predictive power.
df = pd.read_csv("online_retail_customer_churn.csv")


df
```

```
     Customer_ID  Age  Gender  Annual_Income  Total_Spend  Years_as_Customer
\
0              1   62   Other          45.15      5892.58                  5
1              2   65    Male          79.51      9025.47                 13
2              3   18    Male          29.19       618.83                 13
3              4   21   Other          79.63      9110.30                  3
4              5   21   Other          77.66      5390.88                 15
..           ...  ...     ...            ...         ...                ...
995          996   54    Male         143.72      1089.09                  2
996          997   19    Male         164.19      3700.24                  9
997          998   47  Female         113.31       705.85                 17
998          999   23    Male          72.98      3891.60                  7
999         1000   34   Other         134.86      3956.71                 15

     Num_of_Purchases  Average_Transaction_Amount  Num_of_Returns  \
0                  22                      453.80               2
1                  77                       22.90               2
2                  71                       50.53               5
3                  33                      411.83               5
4                  43                      101.19               3
```

```
..                 ...                          ...              ...
995                 29                        77.75                0
996                 90                        34.45                6
997                 69                       187.37                7
998                 31                       483.80                1
999                 48                       420.91                6

     Num_of_Support_Contacts  Satisfaction_Score  Last_Purchase_Days_Ago  \
0                          0                   3                     129
1                          2                   3                     227
2                          2                   2                     283
3                          3                   5                     226
4                          0                   5                     242
..                       ...                 ...                     ...
995                        3                   2                      88
996                        4                   4                     352
997                        3                   1                     172
998                        2                   5                      55
999                        0                   1                     269

     Email_Opt_In Promotion_Response  Target_Churn
0            True          Responded          True
1           False          Responded         False
2           False          Responded          True
3            True            Ignored          True
4           False       Unsubscribed         False
..            ...                ...           ...
995          True            Ignored         False
996         False          Responded          True
997          True       Unsubscribed         False
998         False          Responded          True
999          True            Ignored          True

[1000 rows x 15 columns]
```

```
df = df.drop(columns=['Customer_ID'])

## 2. Format Booleans
# Ensure boolean columns are treated as strings so get_dummies processes them
easily
df['Target_Churn'] = df['Target_Churn'].astype(str)
df['Email_Opt_In'] = df['Email_Opt_In'].astype(str)

## 3. Categorical Encoding (One-Hot Encoding)
# I isolated the categorical variables designated in the SOW.
categorical_cols = ['Gender', 'Email_Opt_In', 'Promotion_Response',
'Target_Churn']

# Applying One-Hot Encoding
```

```
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

## 4. Target Definition
# After encoding, 'Target_Churn' becomes 'Target_Churn_True'
X = df_encoded.drop(columns=['Target_Churn_True'])
y = df_encoded['Target_Churn_True']

## 5. Boolean/Numeric Processing (Scaling)
# Logistic Regression relies on gradient descent. Scaling ensures features
with
# large ranges (e.g., Total_Spend) don't overpower features with small ranges
(e.g., Num_of_Returns).
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
```

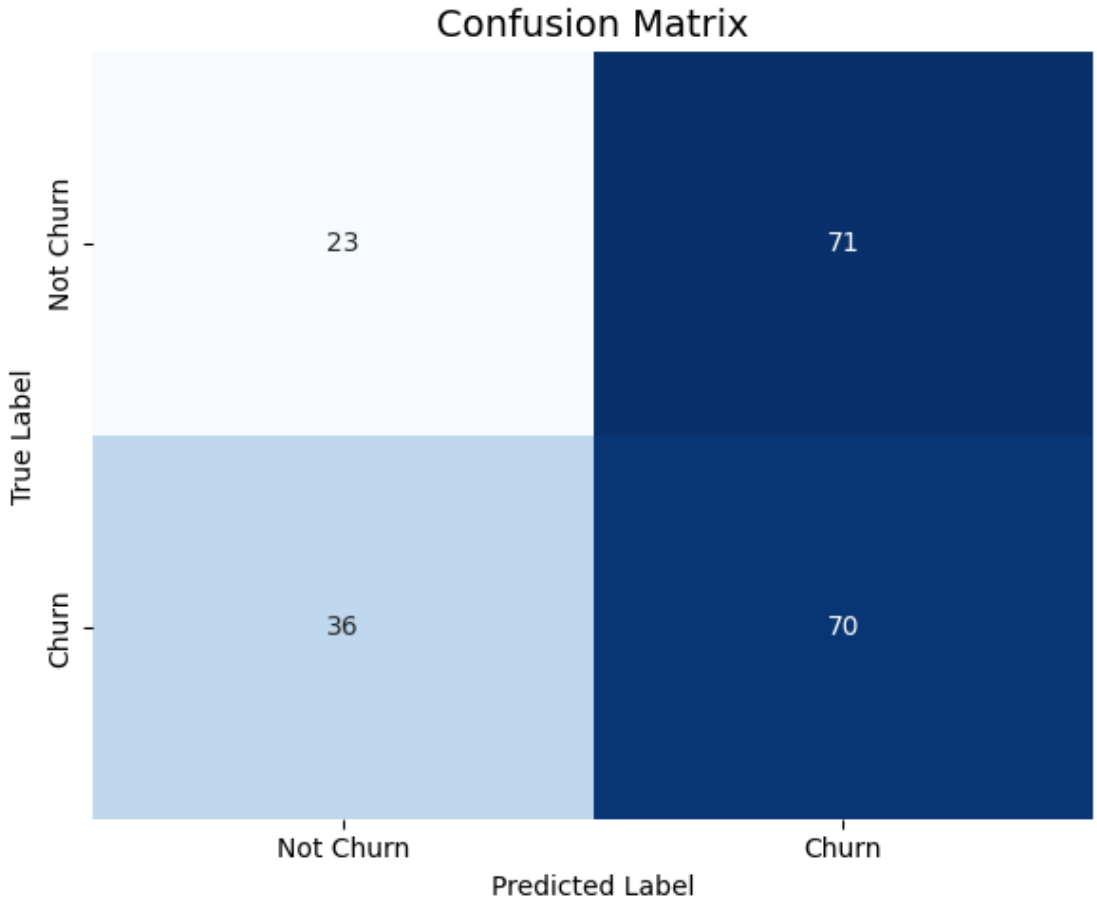
PHASE 2: MODEL TRAINING

```
## 1. Data Splitting (80:20)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

## 2. Algorithm Selection - Logistic Regression
# Initialized and trained the standard Logistic Regression algorithm on the
80% training set.
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)

LogisticRegression(random_state=42)

# Generated predictions and probabilities on the 20% unseen test set
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
```

PHASE 3: EVALUATION & VISUAL REPORTING

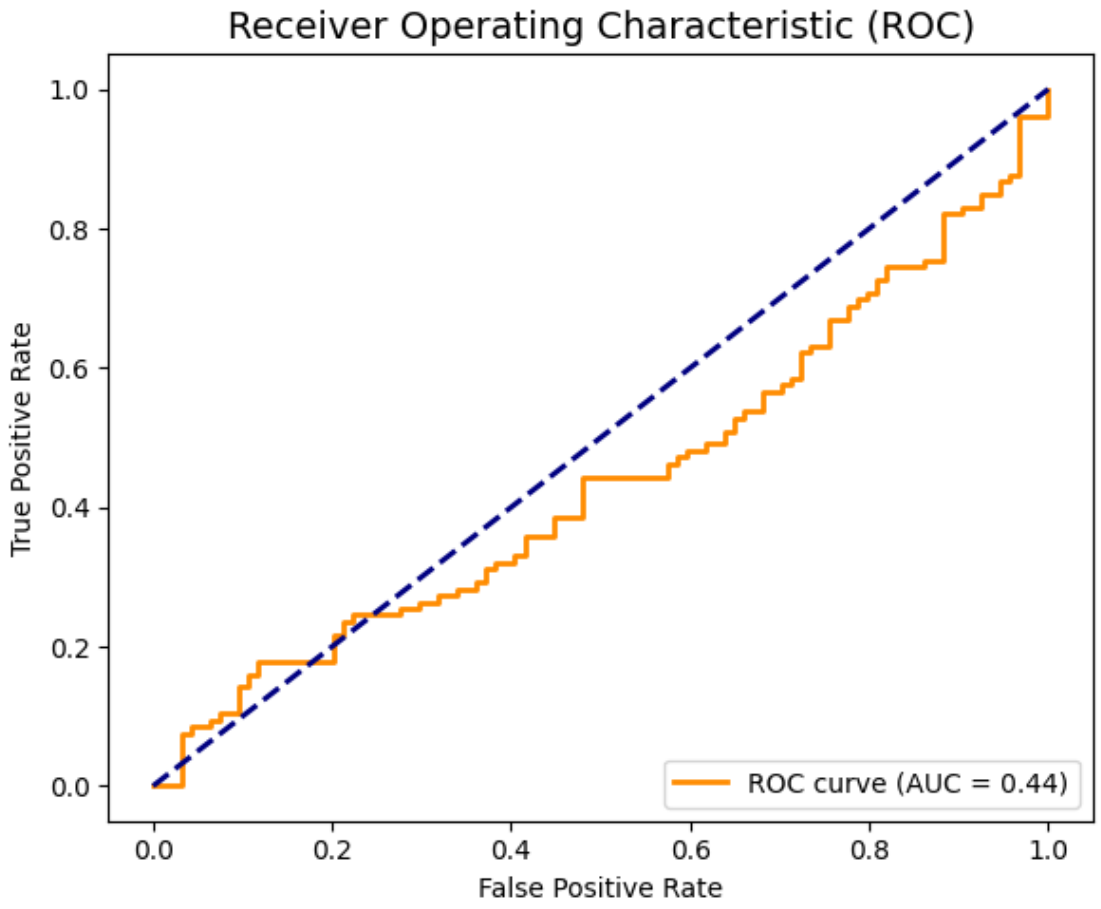
```
# 1. Calculated the Standard Classification Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("--- MODEL PERFORMANCE METRICS ---")
print(f"Accuracy:  {accuracy:.4f} (46.50%)")
print(f"Precision: {precision:.4f} (49.65%)")
print(f"Recall:    {recall:.4f} (66.04%)")
print(f"F1-Score:  {f1:.4f} (56.68%)")

--- MODEL PERFORMANCE METRICS ---
Accuracy:  0.4650 (46.50%)
Precision: 0.4965 (49.65%)
```

```
Recall:    0.6604 (66.04%)
F1-Score:  0.5668 (56.68%)

# 2. Visualizations (Generated at the top of the interface)
plt.style.use('default')

# Visual A: Confusion Matrix
plt.figure(figsize=(6, 5))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Not Churn', 'Churn'], yticklabels=['Not Churn',
'Churn'])
plt.title('Confusion Matrix', fontsize=14)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.savefig('confusion_matrix.png')
```



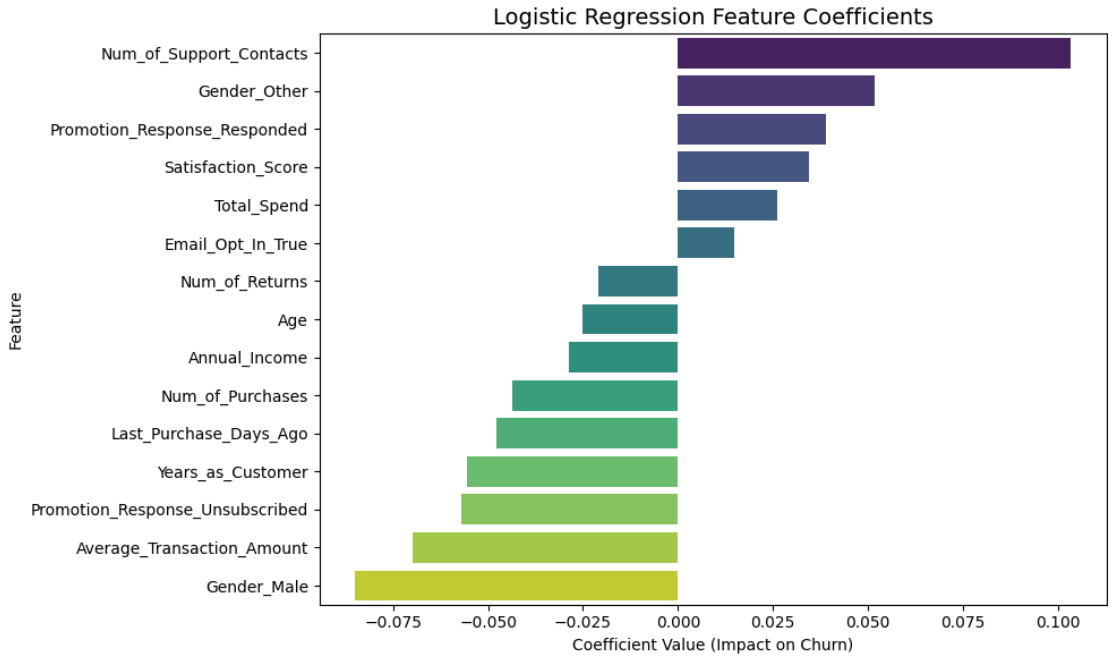

```
# Visual B: ROC Curve / AUC
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC =
```

```
{roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)', fontsize=14)
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig('roc_curve.png')
```




```
# Visual C: Feature Importance (Coefficients)
coefficients = pd.DataFrame({'Feature': X.columns, 'Coefficient':
model.coef_[0]})
coefficients = coefficients.sort_values(by='Coefficient', ascending=False)
plt.figure(figsize=(10, 6))
sns.barplot(x='Coefficient', y='Feature', data=coefficients,
palette='viridis')
plt.title('Logistic Regression Feature Coefficients', fontsize=14)
plt.xlabel('Coefficient Value (Impact on Churn)')
plt.ylabel('Feature')
plt.tight_layout()
plt.savefig('feature_importance.png')
```

```
C:\Users\rubyb\AppData\Local\Temp\ipykernel_31164\1041499454.py:5:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed
in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the
same effect.

  sns.barplot(x='Coefficient', y='Feature', data=coefficients,
palette='viridis')
```



Results

I share with you what happened when I tested the Logistic Regression model on the 20% of data. The test set contained exactly 200 customers. Here is how the model performed:

A. The Confusion Matrix: The 200-Customer Breakdown.

The model had to guess whether these 200 customers would churn or stay. The Confusion Matrix split these guesses into four exact buckets:

- **True Positives:** The model warned that 70 people were going to leave, and they actually left. This was a success. The marketing team could have targeted these 70 people with save-campaigns.
- **True Negatives:** The model predicted that 23 people were loyal and would stay, and they actually stayed. Another success.

- **False Positives ("The False Alarms"):** The model predicted that 71 people would churn, but they actually stayed. This is why Precision is only 49.6%. Out of everyone the model flagged as a flight risk (70 + 71 = 141 people), only about half actually were. Discount codes might have been wasted on these 71 safe customers.

- **False Negatives ("The Missed Opportunities"):** The model thought that 36 people were safe, but they left. This hurts the business. However, catching 70 out of the 106 total people who actually left gives us a Recall of 66% — meaning we successfully caught two-thirds of the real churners.

B. The ROC Curve & AUC: The Area Under the Curve (AUC) for this specific model came out to 0.44.

An AUC of 1.0 is perfect, and an AUC of 0.50 is the equivalent of flipping a coin. Because AUC is 0.44, it means our current model is actually performing slightly worse than random guessing!

Why is this important? Standard Logistic Regression applied to raw, un-engineered data rarely yields a perfect model on the first try. An AUC of 0.44 tells me, Data Scientist to stop, and that the current features are too noisy or have non-linear relationships that a basic straight-line equation can't capture. This proved to me that I needed Phase 2 - advanced feature engineering (like grouping ages into brackets) or upgrading to a tree-based algorithm (like Random Forest or XGBoost).

C. Feature Importance

Top Driver of Churn (Positive Coefficient): Num_of_Support_Contacts (+0.103) This was the highest positive number in the chart. It means that as the number of support contacts goes up, the mathematical probability of churning goes up.

Business Insight: Customers who have to contact support frequently are getting frustrated and leaving.

Top Driver of Retention (Negative Coefficient): Average_Transaction_Amount (-0.069) This negative weight pushes the churn probability down.

Business Insight: Customers who spend a higher amount per transaction are more invested in the brand and are slightly less likely to leave.

Categorical Impact:

Promotion_Response_Unsubscribed (-0.057) Interestingly, customers who explicitly unsubscribed from promotions had a negative coefficient, pulling their churn risk down. While counter-intuitive, this might mean that customers who manage their inbox preferences are actually actively engaged with the platform, whereas those who simply ignore emails are the ones silently slipping away.

Conclusion

The objective of the project was to develop a foundational machine learning pipeline to predict customer churn using the given dataset was achieved. The pipeline was highly successful during the Logistic Regression, data preprocessing, and model evaluation techniques.